TRAFFIC FLOW AND CONGESTION ANALYSIS

Case Study Report

This case study uses Python to analyze traffic counts on multiple roads over different time intervals. It classifies congestion levels, checks road capacity, and identifies the highest recorded traffic count.

1. Problem Statement

Traffic congestion can cause delays and reduce road efficiency. The aim is to create a Python program that processes vehicle counts for several roads at different time intervals, labels congestion as low, medium, or high, reports when traffic exceeds road capacity, and identifies the peak traffic count along with the road and time interval where it occurs.

2. Objective

The objective is to apply Python lists, nested lists, loops, conditional statements, and variables to analyze traffic data, classify congestion, detect capacity exceedance, and find peak traffic.

3. Assumptions

• The roads considered are Road A, Road B, and Road C.

• Traffic data contains three vehicle counts for each road, representing three time intervals.

• The road capacity threshold is 100 vehicles for a time interval.

• More than 100 vehicles is classified as high congestion; 50 to 100 vehicles is medium congestion; fewer than 50 vehicles is low congestion.

• A road is reported as exceeding capacity if any of its recorded intervals has more than 100 vehicles.

• The peak traffic is the largest vehicle count in the complete data set, and the program records its road and time interval.

4. Algorithm

1. Start the program and define the road names and traffic counts for each time interval.

2. Set road capacity to 100 and initialize the peak traffic count to 0.

3. For each road, display its name and examine each time interval.

4. Read the vehicle count and compare it with the current peak; if it is larger, update the peak count, road, and time interval.

5. Classify the count as high congestion if it is greater than 100, medium congestion if it is at least 50, or low congestion otherwise.

6. After checking a road's intervals, print a capacity warning if its maximum count is greater than 100.

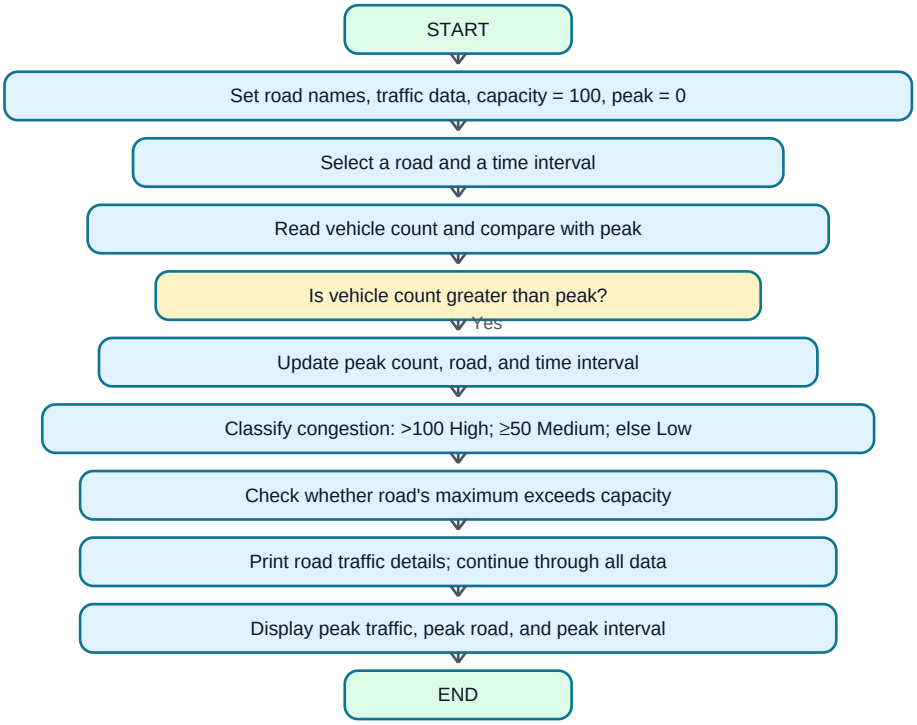
7. Repeat until all roads and time intervals have been processed.

8. Display the peak traffic count, the road with peak traffic, and the time interval.

9. Stop the program.

5. Flowchart

The flowchart summarizes the main processing steps used to classify traffic and identify peak congestion.



6. Pseudocode

```
BEGIN
    roads ← ["Road A", "Road B", "Road C"]
    traffic ← [[40, 60, 90], [30, 80, 110], [20, 40, 50]]
    capacity ← 100
    peak ← 0

    FOR each road i
        PRINT roads[i]
        FOR each time interval j
            vehicles ← traffic[i][j]

            IF vehicles > peak THEN
                peak ← vehicles
                peak_road ← roads[i]
                peak_time ← j + 1
            END IF

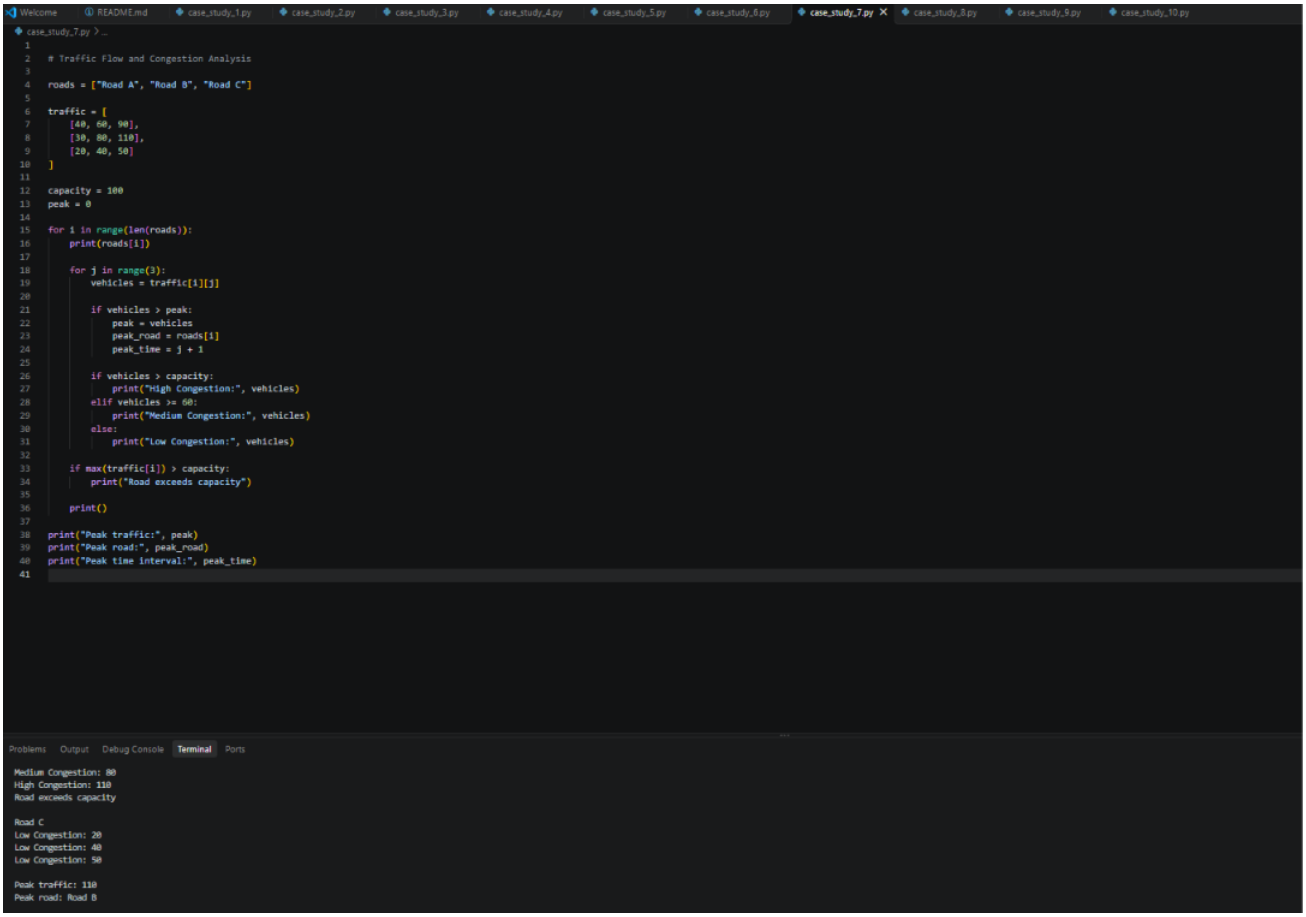
            IF vehicles > capacity THEN
                PRINT "High Congestion"
            ELSE IF vehicles >= 50 THEN
                PRINT "Medium Congestion"
            ELSE
                PRINT "Low Congestion"
            END IF
        END FOR

        IF maximum traffic for road i > capacity THEN
            PRINT "Road exceeds capacity"
        END IF
    END FOR

    PRINT peak, peak_road, peak_time
END
```

7. Screenshot of Python Source Code and Output

The original screenshot of the Python source code and terminal output is included below. The displayed results identify 110 vehicles as the peak traffic count on Road B.



8. Conclusion

The Traffic Flow and Congestion Analysis program processes traffic counts across multiple roads and time intervals. It classifies congestion, warns when a road exceeds its capacity, and tracks the highest traffic count. In the supplied data, peak traffic is 110 vehicles on Road B during the third time interval. This case study demonstrates how basic Python programming can help summarize traffic conditions and highlight potential congestion points.